

I. Compiling a custom Linux Kernel

● Screenshot

```
~ #  
~ # uname -a  
Linux (none) 6.1.0-os-a141064 #3 SMP Sun Nov 23 06:51:54 UTC 2025 riscv64 GNU/Linux  
~ # uname -a  
Linux (none) 6.1.0-os-a141064 #3 SMP Sun Nov 23 06:51:54 UTC 2025 riscv64 GNU/Linux  
~ # ./test_revstr  
Ori: hello  
Rev: olleh  
Ori: Operating System  
Rev: metsyS gnitarepO  
~ # ./test_temdbuf  
Hello Operating Systems  
Operating Systems  
~ #
```

● The steps to compile the kernel

- 實作核心程式碼 (Source Code) 在 `kernel/` 目錄下建立 `sys_revstr.c` 與 `sys_temdbuf.c`。
- `sys_revstr`：利用 `kmalloc` 配置記憶體，透過 `copy_from/to_user` 傳輸資料，並使用原地交換演算法反轉字串。
- `sys_temdbuf`：使用核心內建的 `Linked List` 管理資料，並加入 `Mutex` 鎖 保護全域串列，防止多執行緒並發導致的 `Race Condition`。
- 註冊系統呼叫編號 (Syscall ID) 修改 `include/uapi/asm-generic/unistd.h` (RISC-V 通用架構)。
- 定義 `__NR_revstr` 為 451。
- 定義 `__NR_temdbuf` 為 452。
- 更新 `__NR_syscalls` 總數為 453。

- 宣告函數原型 (Prototypes) 修改 `include/linux/syscalls.h`，加入 `asmlinkage` 函數宣告，確保編譯器能正確處理參數傳遞與型別檢查。修改編譯設定 (Makefile) 修改 `kernel/Makefile`，在 `obj-y` 列表中加入 `sys_revstr.o` 與 `sys_tempbuf.o`，指示編譯系統將這兩個新檔案編譯並連結至 `Kernel Image` 中。

- ## Q&As

Q1: What are the main differences between RISC-V and x86 architectures?

A: RISC-V is an open-source, modular RISC architecture focused on simplicity and flexibility, while x86 is a proprietary CISC architecture controlled by Intel and AMD, known for complex instructions and a mature ecosystem.

Q2: Why do the architecture differences matter when building the kernel? What happens if you build the kernel without the correct RISC-V cross-compilation flag?

A: The kernel must match the target hardware's instruction set; if you build without the correct RISC-V cross-compilation flag, the kernel will not run on RISC-V hardware and will fail to boot or operate properly.

Q3: Why is Docker used in this assignment, and what advantages does it provide? Please list at least two.

A: Docker provides a consistent, isolated build environment for all students, making setup and troubleshooting easier, and ensures everyone uses the same toolchain and dependencies for reproducible results.

II. Implementing new System Calls

- Include four screenshots of the outputs:

- Screenshot 1: Execution result of test_revstr

```
~ # ./test_revstr
Ori: hello
Rev: olleh
Ori: Operating System
Rev: metsyS gnitarepO
~ #
```

Screenshot 2: dmesg output of sys_revstr

```
[ 3.944281] sys_revstr entered!
[ 3.944347] The origin string: hello
[ 3.944378] The reversed string: olleh
[ 3.944993] sys_revstr entered!
[ 3.945005] The origin string: Operating System
[ 3.945013] The reversed string: metsyS gnitarepO
```

(sys_revstr entered! is for debug)

Screenshot 3: Execution result of test_tempbuf

```
~ # ./test_tempbuf
Hello Operating Systems
Operating Systems
~ #
```

Screenshot 4: dmesg output of sys_tempbuf

```
[ 122.317145] [tempbuf] Added: Hello
[ 122.317322] [tempbuf] Added: Operating Systems
[ 122.317430] [tempbuf] Hello Operating Systems
[ 122.318939] [tempbuf] Removed: Hello
[ 122.319000] [tempbuf] Operating Systems
```

- Explain how you added system calls to your custom Linux kernel and the implementation details of sys_revstr and sys_tempbuf

I edited arch/riscv/include/uapi/asm/unistd.h and appended custom syscall numbers and their mappings.

```
Ubuntu > home > a141064 > linux > arch > riscv > include > uapi > asm > C unistd.h
 1  /* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
 2  /*
 3   * Copyright (C) 2018 David Abdurachmanov <david.abdurachmanov@gmail.com>
 4   *
 5   * This program is free software; you can redistribute it and/or modify
 6   * it under the terms of the GNU General Public License version 2 as
 7   * published by the Free Software Foundation.
 8   *
 9   * This program is distributed in the hope that it will be useful,
10   * but WITHOUT ANY WARRANTY; without even the implied warranty of
11   * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
12   * GNU General Public License for more details.
13   *
14   * You should have received a copy of the GNU General Public License
15   * along with this program. If not, see <https://www.gnu.org/licenses/>.
16   */
17
18 #if defined(__LP64__) && !defined(__SYSCALL_COMPAT)
19 #define __ARCH_WANT_NEW_STAT
20 #define __ARCH_WANT_SET_GET_RLIMIT
21 #endif /* __LP64__ */
22
23 #define __ARCH_WANT_SYS_CLONE3
24 #define __ARCH_WANT_MEMFD_SECRET
25
26 #include <asm-generic/unistd.h>
27
28 /*
29  * Allows the instruction cache to be flushed from userspace. Despite RISC-V
30  * having a direct 'fence.i' instruction available to userspace (which we
31  * can't trap!), that's not actually viable when running on Linux because the
32  * kernel might schedule a process on another hart. There is no way for
33  * userspace to handle this without invoking the kernel (as it doesn't know the
34  * thread->hart mappings), so we've defined a RISC-V specific system call to
35  * flush the instruction cache.
36  *
37  * __NR_riscv_flush_icache is defined to flush the instruction cache over an
38  * address range, with the flush applying to either all threads or just the
39  * caller. We don't currently do anything with the address range, that's just
40  * in there for forwards compatibility.
41  */
42 #ifndef __NR_riscv_flush_icache
43 #define __NR_riscv_flush_icache (__NR_arch_specific_syscall + 15)
44 #endif
45 __SYSCALL(__NR_riscv_flush_icache, sys_riscv_flush_icache)
46 #undef __NR_syscalls
47 #define __NR_syscalls 453
48
49 #define __NR_revstr 451
50 #define __NR_tempbuf 452
51 __SYSCALL(__NR_revstr, sys_revstr)
52 __SYSCALL(__NR_tempbuf, sys_tempbuf)
```

- `__NR_revstr` and `__NR_tempbuf` use unique, non-conflicting numbers above the default.
- Redefining `__NR_syscalls` ensures the kernel's syscall table (`sys_call_table[]`) is large enough to index up to the highest custom syscall number (452), thus preventing out-of-bounds errors.
- `#undef` is essential: it removes the earlier value (e.g., 451 from `asm-generic/unistd.h`) so your custom value takes effect cleanly.

The implementation of **`sys_revstr`** involves copying a string from user space into a kernel buffer, reversing the string in place, and then copying the reversed string back to user space. The function checks the input length for safety, uses `copy_from_user` and `copy_to_user` for secure data transfer, and logs both the original and reversed strings to the kernel log for debugging.

The **`sys_tempbuf`** system call manages a kernel-side linked list of strings, supporting three modes: ADD, REMOVE, and PRINT. In ADD mode, it allocates memory for a new string and appends it to the list; in REMOVE mode, it searches for and deletes a matching string from the list; in PRINT mode, it concatenates all stored strings into a buffer, logs the result, and copies it back to user space. The implementation uses kernel synchronization primitives (mutex) to ensure thread safety and provides detailed kernel log output for each operation, making it easy to verify correct behavior with `dmesg`.

● Answer the Q&As

1. What does `include/linux/syscalls.h` do?

It lists all the system call functions in the kernel, so the kernel

knows what functions are available for system calls.

2. System call vs glibc library call (with example):

A system call lets a program ask the kernel to do something, like read a file.

A glibc library call is a user-space function that often wraps a system call.

3. Static linking vs dynamic linking:

Static linking puts all needed library code into the program when you build it, so the program runs anywhere.

Dynamic linking uses shared libraries at run time, so the program is smaller but needs those libraries to be present.

4. Why use -static for test programs? What if omitted?

You need -static because the test system has no shared libraries.

If you don't use it, your program won't run—it will fail with a missing library error.